

Lernziele:

- `malloc/free`:
 - Anzahl der zu reservierenden Bytes mit `sizeof` festlegen
 - Memory-Leaks, mehrfache Freigabe
 - Ungültige Pointer (*Null-Pointer*)
 - Pointer auf einen unbekannten Datentypen (*Void-Pointer*)
 - Fehlerbehandlung
- Struct für Listen
- Installation ULM-Generator

Schritt 1: Ungültige Zeiger

In folgendem Programm werden nicht-initialisierte Zeiger dereferenziert:

```
1 @ <stdio.h>
2 @ <stdlib.h>
3
4 global q: -> int;
5
6 fn main()
7 {
8     local p: -> int;
9
10     *p = 4;
11     *q = 5;
12 }
```

Das Programm lässt sich kompilieren, aber wenn es ausgeführt wird, stürzt es mit einem Fehler "Segmentation Fault" (was in der Regel den ausgestreckten Mittelfinger des Betriebssystems beschreibt) ab.

Aufgaben:

1. Gib jeweils den Wert der Zeigervariablen `p` und `q` aus, bevor diese dereferenziert werden.
2. Wird die Zuweisung in Zeile 11 auskommentiert, dann stürzt das Programm möglicherweise nicht ab. Erkläre, wieso es schlecht ist, dass dann das Programm nicht abstürzt.

Schritt 2: Null-Zeiger

Ein Zeiger ist genau dann gültig, wenn er auf eine allokierte (d.h. reservierte) Speicherfläche zeigt. Das sind Speicherflächen von globalen Variablen, lokalen Variablen (wenn diese noch im gültigen Bereich des Stacks liegen) und Speicherflächen, die dynamisch (z.B. mit `malloc`) allokiert wurden und noch nicht freigegeben wurden.

Um stets ein definiertes Verhalten zu erzielen, sollten lokale Zeigervariablen entweder mit einer gültigen Adresse oder der Konstanten `nullptr` initialisiert werden. Die Konstante `nullptr` in C, C++ und ABC ist eine Adresse, bei der garantiert wird, dass es zu einem Programmabsturz kommt, wenn ein Zeiger mit diesem Wert dereferenziert wird.

Aufgabe: Initialisiere im Programm der vorigen Aufgabe die lokale Variable `p` mit dem Wert `nullptr`. Bestätige, dass das Programm jetzt beim Dereferenzieren von `p` abstürzt.

Bemerkung: Ob `nullptr` tatsächlich den Wert `0` hat ist Plattform abhängig.

Schritt 3: Pattern für den Aufruf von malloc

Mit einem Aufruf von `malloc` kann dynamisch Speicher allokiert werden. Dies kann aber fehlschlagen, weil nicht ausreichend Speicher zur Verfügung steht. In diesem Fall gibt `malloc` den Wert `nullptr` zurück. Nach jedem Aufruf von `malloc` sollte deshalb der Rückgabewert überprüft werden, und das Programm sollte mit einem Exit-Code ungleich `0` terminiert werden. Folgendes Programm bricht in diesem Fall mit einer Fehlermeldung und dem Exit-Code `1` ab (die Funktion `exit` ist in `<stdlib.h>` deklariert):

```

1 @ <stdio.h>
2 @ <stdlib.h>
3
4 fn main()
5 {
6     local p: -> int = malloc(4);
7     if (p == nullptr) {
8         printf("out of memory\n");
9         exit(1);
10    }
11 }
```

Aufgaben:

1. Übersetze das obige Programm und führe es aus. Frage anschließend den Exit-Code mit dem Shell-Kommando `echo $?` ab.
2. Die übliche Konvention ist, dass ein Exit-Code von `0` eine erfolgreiche Ausführung anzeigt. Ändere aber für Testzwecke das Programm so ab, dass es bei einer erfolgreichen Ausführung den Exit-Code `42` liefert.
3. In Zeile 7 wurde mit `p == nullptr` überprüft, ob `p` ein gültiger Zeiger ist oder nicht. Man muss dazu den Wert von `p` aber nicht explizit mit `nullptr` vergleichen. Wird `p` in einem logischen Ausdruck verwendet, dann wird der Wert von `nullptr` als `false` und jeder andere Wert als `true` interpretiert. Ändere die logische Bedingung in Zeile 7 so ab, dass `nullptr` nicht verwendet wird.
4. Der Funktion `malloc` übergibt man als Parameter die Größe der Speicherfläche, die allokiert werden soll. Diese Größenangabe ist in Bytes. Im obigen Beispiel wurde der Wert `4` verwendet, da dies auf vielen Plattformen die Größe des Datentyps `int` ist. Plattformunabhängig (und damit besser) wäre es, die tatsächliche Größe vom Compiler in den Code einsetzen zu lassen, indem man den `sizeof`-Operator verwendet:

```
6     local p: -> int = malloc(sizeof(int));
```

Nun besteht allerdings die Gefahr, dass man später den Typ von `p` ändert und dabei vergisst, beim `sizeof`-Operator den Typ anzupassen:

```
6     local p: -> u64 = malloc(sizeof(int));
```

Löse dieses Problem! Wende `sizeof` so an, dass immer die Größe der Speicherfläche am Ende von `p` an `malloc` übergeben wird, ohne den Datentyp explizit zu verwenden.

5. In folgendem Programm wurde `p` als globale Zeigervariable definiert:

```

1 @ <stdio.h>
2 @ <stdlib.h>
3
4 global p: -> int = malloc(sizeof(*p));
5
6 fn main()
7 {
8     if (p == nullptr) {
9         printf("out of memory\n");
10        exit(1);
11    }
12 }
```

Allerdings weigert sich der Compiler, dieses Programm zu übersetzen. Die Initialwerte von globalen Variablen müssen zur Compile-Zeit bekannt sein. Erkläre, wieso das hier nicht der Fall ist. Ändere das Programm dann so ab, dass `p` zur Laufzeit mit `malloc` ein Wert zugewiesen wird.

Schritt 4: Memory-Leaks und Verwendung von free

Wird mit `malloc` Speicher allokiert, ist dieser solange reserviert, bis das Programm terminiert oder der Speicher mit `free` freigegeben wird. Um Memory-Leaks (Speicherlecks) zu vermeiden, sollte für jeden Aufruf von `malloc` sichergestellt werden, dass auch ein zugehöriges `free` aufgerufen wird. Dabei sollten folgende Regeln beachtet werden:

- A. `free` erhält als Argument genau den Wert, den `malloc` zurückgegeben hat.
- B. `free` wird mit diesem Wert genau einmal aufgerufen.

Folgendes Programm lässt sich übersetzen, verletzt jedoch beide Regeln:

```

1 @ <stdio.h>
2 @ <stdlib.h>
3
4
5 fn main()
6 {
7     local p: -> int = malloc(sizeof(*p));
8     if (p == nullptr) {
9         printf("out of memory\n");
10        exit(1);
11    }
12
13    ++p;
14    free(p);
15    free(p);
16 }
```

Ob das Programm bei der Ausführung abstürzt, ist nicht garantiert. Das Laufzeitverhalten ist undefiniert. Überprüfe das Laufzeitverhalten für folgende Fälle:

1. Übersetze das Programm wie angegeben und führe es aus.
2. Kommentiere Zeile 15 aus.
3. Entferne den Kommentar in Zeile 15, kommentiere dafür aber Zeile 13 aus.

Vergleiche deine Ergebnisse mit den Ergebnissen anderer! Insbesondere mit den Ergebnissen auf einer anderen Plattform (z.B. anderes Betriebssystem).

Bemerkung

Zwei Dinge sind erstaunlich:

1. Mit `malloc` kann man Speicher für unterschiedliche Datentypen generieren.
2. Bei `free` muss nur die Anfangsadresse einer Speicherfläche, aber nicht deren Größe, angegeben werden.

Hier eine kurze Erklärung für beides:

1. Der Rückgabety von `malloc` ist vom Typ `-> void`, was oft als *Void-Pointer* bezeichnet wird. Damit ist aber kein ungültiger Zeiger gemeint, sondern ein Zeiger auf einen nicht festgelegten Typen. In der Buchhaltung des Compilers ist dieser sozusagen als *unbekannt* vermerkt. Dieser Typ kann in einen beliebigen anderen Pointer-Typ (implizit oder explizit) gecastet werden. Der Programmierer ist verantwortlich, dass dieser Cast sinnvoll ist (also am Ende des Zeigers gültiger Speicher liegt).
2. Die Funktionen `malloc/free` können unterschiedlich implementiert werden. Eine Möglichkeit ist, dass `malloc` stets etwas mehr Speicher allokiert und in dem zusätzlichen Speicher Verwaltungsinformationen wie die Blockgröße ablegt. Auf diese Information kann `free` zugreifen. Werden beispielsweise `k` Bytes mit `malloc` erfolgreich reserviert und der Rückwert in `p` gespeichert, liegen am Ende von `p` diese Informationen links und der angeforderte Block rechts.



Schritt 5: Eine erste einfach verkettete Listen

Bevor wir in diesem Schritt auf die technischen Details eingehen, übersetzt folgendes Programm und führt es aus:

```
1 @ <stdio.h>
2
3 struct Node
4 {
5     val: array[8] of char;
6     next: -> Node;
7 };
8
9 global a, b, c: Node =
10 {
11     {"world", &b},
12     {"hello", &c},
13     {"!"},
14 };
15
16 fn main()
17 {
18     for (local p: -> Node = &a; p; p = p->next) {
19         printf("%s ", p->val);
20     }
21     printf("\n");
22 }
```

Es ist zunächst erstaunlich, dass der Compiler diesen Code akzeptiert, denn in den Zeilen 3-7 wird ein Datentyp `Node` definiert, der innerhalb dieser Definition (in Zeile 6) selbst auftaucht. Dass der Compiler diese Definition aber akzeptiert, kann man vereinfacht so erklären:

- Ab Zeile 3 ist dem Compiler bekannt, dass es einen Datentyp `Node` geben wird. Die Größe ist aber erst am Ende der Definition (also nach Zeile 7) bekannt.
- Der Compiler akzeptiert eine Struct-Deklaration, wenn er die Größe für jedes Member kennt. Member `next` wird in Zeile 6 als Zeiger auf den Datentyp `Node` vereinbart. Die Größe eines Zeigers ist unabhängig davon, wie groß der Datentyp am Ende des Zeigers ist.

Aufgaben:

- In Zeile 9 werden die Variablen `a`, `b` und `c` definiert und initialisiert. Skizziere die zugehörigen Speicherflächen dieser Variablen und deren Inhalt.
- Erkläre welche Werte die lokale Variable `p` beim Durchlaufen der For-Schleife in den Zeilen 18-20 annimmt.
- Passe Zeile 18 sowie in Zeile 11-13 die Initialisierung der jeweiligen `next`-Member so an, dass das Programm `hello, world!` ausgibt.